Analisi delle funzioni di hashing di una tabella hash con indirizzamento a concatenazione

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 31, 2024
Studiato	Done

Indichiamo con U l'insieme universo delle chiavi. Indichiamo con K le chiavi memorizzate. Con $T[0, ..., n-1]$ si indica la tabella di indirizzamento diretto, con $n = |K|$.

La funzione hash è la funzione del tipo:

$$f : U \to \{\, 0, ..., m-1 \,\}$$

Quindi data una chiave k, avremo che:

$$f(k) \in \{\, 0, ..., m-1 \,\}$$

Abbiamo definito il fattore di carico α come la dimensione media di una lista di posizione $T[i]$:

$$\alpha = \frac{n}{m}$$

Il valore α si avvicina maggiormente alla realtà dei fatti quando la funzione di hashing è **indipendente e uniforme**.

Supponiamo che $|T[j]| = n_j$. Sia $n_0 + n_1 + ... + n_{m-1} = n$, ossia la somma del numero di elementi in ogni lista è uguale al numero n di chiavi inserite. Assumiamo che **il calcolo di $h(k)$ e l'accesso allo slot corrispondente $T[h(k)]$ richiedano tempo costante $\Theta(1)$.**

Utilizzeremo il valore atteso poiché abbiamo la probabilità che una chiave k attraverso h abbia immagine uguale a j. Tale probabilità dipende chiaramente dalla funzione hash. Tale analisi, quindi, prenderà in considerazione una funzione hash **uniforme** e **indipendente** (indipendente indica il fatto che **se** $h(k_1) = j$**, allora** $h(k_2)$ **non deve dipendere da** $h(k_1)$).

Valore atteso della ricerca senza successo

> *Il valore atteso del tempo computazionale di una ricerca senza successo su una tabella hash dove le collisioni sono risolte per concatenazione è* $\Theta(1 + \alpha)$.

Dimostriamolo. Il tempo della ricerca è dato dal tempo per calcolare $h(k)$, il tempo per accedere a $T[h(k)]$ e il tempo per controllare ogni valore nella lista concatenata. Teniamo conto del **valore atteso del numero di elementi nella lista concatenata**:

$$\mathbb{E}[T] = \Theta(1) + \mathbb{E}[n_{h(k)}] = \Theta(1) + \sum_{j=0}^{m-1} P_r[h(k) = j] \cdot n_j =$$

$$= \Theta(1) + \frac{1}{m} \sum_{j=0}^{m-1} n_j = \Theta(1) + \frac{n}{m} = \Theta(1) + \alpha =$$

$$= \Theta(1 + \alpha)$$

Valore atteso della ricerca con successo

> *Il valore atteso del tempo computazionale di una ricerca con successo su una tabella hash dove le collisioni sono risolte per concatenazione è* $\Theta(1 + \alpha)$.

Il tempo della ricerca è dato sempre dal tempo per calcolare $h(k)$, il tempo per accedere a $T[h(k)]$ e il tempo per controllare il numero di elementi che hanno lo stesso hash e sono stati **inseriti dopo** l'elemento cercato (supponendo un inserimento in testa).

Supponiamo che x_i sia l'elemento che stiamo cercando. Sia x_1 l'elemento inserito per primo, x_2 il secondo e così via. Ne segue che x_i è l'i-esimo elemento inserito nella tabella.

Sia $k_i = x_i.key$, e $\forall q \in \{0, ..., m-1\}, \forall k_i, k_j \in K, k_i \neq k_j$ consideriamo la variabile:

$$X_{ijq} = \begin{cases} 1 & \text{se } x_i \text{ è l'elemento cercato}, h(k_i) = q, h(k_j) = q \\ 0 & \text{altrimenti} \end{cases}$$

Quindi $X_{ijq} = 1$ se le chiavi k_i, k_j collidono nella stessa cella q e stiamo cercando l'elemento x_i. Allora possiamo dire che:

$$\mathbb{E}[X_{ijq}] = 0 \cdot P_r[X_{ijq} = 0] + 1 \cdot P_r[X_{ijq} = 1] = \\ = P_r[x_i \text{ è l'elemento cercato}] \cdot P_r[h(k_i) = q] \cdot P_r[h(k_j) = q]$$

Usando l'ipotesi di uniformità abbiamo:

$$\frac{1}{n} \cdot \frac{1}{m} \cdot \frac{1}{m} = \frac{1}{nm^2}$$

Definiamo un'altra variabile:

$$Y_j = \begin{cases} 1 & \text{se } x_j \text{ appare nella stessa lista e prima dell'elemento cercato} \\ 0 & \text{altrimenti} \end{cases}$$

Poiché non stiamo facendo riferimento ad un preciso spazio in T, allora:

$$Y_j = \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq}$$

Notiamo che il valore $X_{ijq} = 1$ solo per una scelta di q, i, infatti è pari ad 1 quando l'elemento x_i appartiene alla lista puntata da q, in cui è presente x_j e se $i < j$, cioè x_i appare dopo x_j.
Infine definiamo una variabile
Z tale che:

$$Z = \sum_{j=1}^{n} Y_j$$

ossia è il **numero di elementi che precedono l'elemento x_i in $T[h(k)]$**. A questo punto possiamo dire che:

$$\mathbb{E}(T) = \Theta(1) + \mathbb{E}[Z+1] = \Theta(1) + 1 + \mathbb{E}[\sum_{j=1}^{n} Y_j] = \Theta(1) + \mathbb{E}[\sum_{j=1}^{n}\sum_{q=0}^{m-1}\sum_{i=1}^{j-1} X_{ijq}] =$$

$$= \Theta(1) + \mathbb{E}[\sum_{q=0}^{m-1}\sum_{j=1}^{n}\sum_{i=1}^{j-1} X_{ijq}] = \Theta(1) + \sum_{q=0}^{m-1}\sum_{j=1}^{n}\sum_{i=1}^{j-1}\mathbb{E}[X_{ijq}] =$$

$$= \Theta(1) + \sum_{q=0}^{m-1}\sum_{j=1}^{n}\sum_{i=1}^{j-1}\frac{1}{nm^2} = \Theta(1) + m \cdot \frac{n(n-1)}{2} \cdot \frac{1}{nm^2} = 1 + \frac{n-1}{2m} =$$

$$= 1 + \frac{n}{2m} - \frac{1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} = \Theta(1+\alpha)$$

poiché per cercare l'elemento, dobbiamo contare l'elemento che stiamo cercando assieme al valore atteso di tutti quelli che lo precedono nella sua lista: quest'ultima quantità è espressa da Z. $\mathbb{E}[Z+1] = \mathbb{E}[Z]$ perché $\mathbb{E}[1] = 1$ e si somma alle costanti.

Osservazione

Se n è al più lineare su m, si ha che $n = O(m) \implies \alpha = \frac{n}{m} \leq \frac{cm}{m} = c$. Quindi $\alpha = \Theta(1)$ e la ricerca impiega $\Theta(1) - Time$.

Grafi

Si definisce grafo una coppia $G = (V, E)$, dove:

- V è l'insieme di vertici;
- $E \subseteq V^2$ ed è l'insieme di archi di G.

Nel caso del grafo non orientato, E contiene sottoinsiemi di cardinalità 2; se il grafo è orientato, E contiene coppie ordinate di vertici. Gli archi (v_1, v_1) sono noti come **cappi** o **self-loop**.

G si dice **completo** se $E = V^2$. Diremo che G è **sparso** se $|E| = o(|V|^2) = O(|V|)$, e diremo che è **denso** se $|E| = \Theta(n^2)$.

Possiamo rappresentare un grafo in due modi:

- **liste di adiacenza**, ossia ogni nodo corrisponde ad una lista che contiene i nodi a lui adiacenti. Occupa $O(|V| + |E|)$ spazio;
- **matrice di adiacenza**, ossia il grafo è una matrice $|V| \times |V|$, dove $e_{ij} = 1$ se vi è un arco da i a j e 0 viceversa. Se G è non orientato, la matrice è simmetrica; se G non ha cappi, la diagonale principale è costituita da soli 0.

La ricerca nelle liste di adiacenza impiega $O(|V|)$, nella matrice $O(1)$.